IT112 – WEB SYSTEMS TECHNOLOGIES
BSIT-2B 2nd Semester

Campus Service Hub:
A Student Guide and Walkthrough in Bicol University Polangui

Group Name:
Code: <breakers>

Members:
Carilla, Grace Ann S.
Lamud, Ella Mae C.
Olarcos, Joshua L.
Requio, Rein Ashley S.
Rubio, Ma. Miscy M.
Sentillas, Tiffany C.

Professor
Guillermo V. Red, Jr

May 2026

Overview of Form Submission and Data Insertion

Phase 4 marks the first concrete integration between the frontend user interface and the backend server of the Campus Service Hub. Having established a functional REST API and a structured frontend in the preceding phases, this phase focused on bridging the two by wiring HTML forms to their corresponding backend routes through JavaScript's fetch() API. The primary objective was to enable real-time data submission from the browser directly into the MongoDB Atlas database, completing the initial input layer of the system.

The group implemented form submission logic across the registration and other relevant pages, ensuring that each input field captured the correct data, validated it on the client side, and transmitted it as a JSON payload to the appropriate POST endpoint. Responses from the backend were handled programmatically, with success and error messages surfaced to the user through browser alerts and console feedback.

Task 1: Verification of POST Routes

Before establishing the frontend-to-backend connection, the group first confirmed that all necessary API endpoints were operational. The backend POST routes established during Phase 3 were re-tested using Postman and Thunder Client to verify that they accepted JSON payloads and returned appropriate HTTP responses. The following routes were confirmed functional prior to frontend integration:

- POST /api/users – Accepts user registration data and stores a new document in the users collection.
- POST /api/products – Accepts product data and inserts a corresponding record into the products collection.

Each test was performed by sending a raw JSON body through the API testing tool, and the responses were validated for correct status codes (200 OK or 201 Created) and the expected data structure. This verification step ensured that the backend was fully ready to receive data from the frontend without errors.

Task 2: Frontend Form Setup

The HTML forms were structured to capture all fields required by the data schema defined in the ERD. Each form element was assigned a unique id attribute to allow JavaScript to precisely reference and extract the input values at the time of submission. The register.html page, for instance, contained fields for the user's name, email address, password, and role, all of which correspond directly to the attributes in the User model on the backend.

An external JavaScript file was linked to each form page via a script tag, keeping the form logic modular and separated from the HTML markup. The default form submission behavior was suppressed using event.preventDefault() to allow the fetch() function to handle data transmission asynchronously, preventing unnecessary page reloads during the process.

Task 3: Form Submission Logic

The form submission handlers were written in dedicated JavaScript files housed within the /js/ subdirectory of the frontend. Each handler followed a consistent pattern: capturing input values from the DOM, constructing a JSON object from those values, and dispatching an asynchronous POST request to the relevant backend endpoint using the fetch() API.

The request was configured with the appropriate headers, specifying the Content-Type as application/json to ensure the backend could correctly parse the incoming payload. Upon receiving the response, the handler evaluated the HTTP status to determine whether the operation was successful. A success response triggered a confirmation alert for the user, while an error response displayed a descriptive message drawn from the backend's JSON reply, allowing the user to understand what went wrong and take corrective action.

Client-side input validation was also applied before the request was dispatched. Required fields were checked for completeness, and basic format validation was enforced on the email field to prevent malformed data from reaching the database. This validation layer reduces unnecessary API calls and improves the overall reliability of the data insertion process.

Task 4: Integration Testing

Live testing was conducted by simultaneously running the frontend through Live Server and the backend server using Nodemon. Forms were submitted with both valid and intentionally invalid data to observe the system's behavior under different conditions. Successful submissions resulted in the expected confirmation alerts on the browser and the creation of new documents in the MongoDB Atlas database, which were verified by inspecting the corresponding collection through the Atlas dashboard.

Browser developer tools were used throughout testing to monitor network requests and inspect console output, providing visibility into the fetch() call lifecycle and the exact request-response exchange between the frontend and backend. Any discrepancies between the expected and actual behavior were identified, debugged, and resolved before the final commit.

CORS configuration on the backend was also verified to be properly set up using the cors() middleware, which was essential in allowing the browser to communicate with the local backend server without being blocked by cross-origin restrictions.

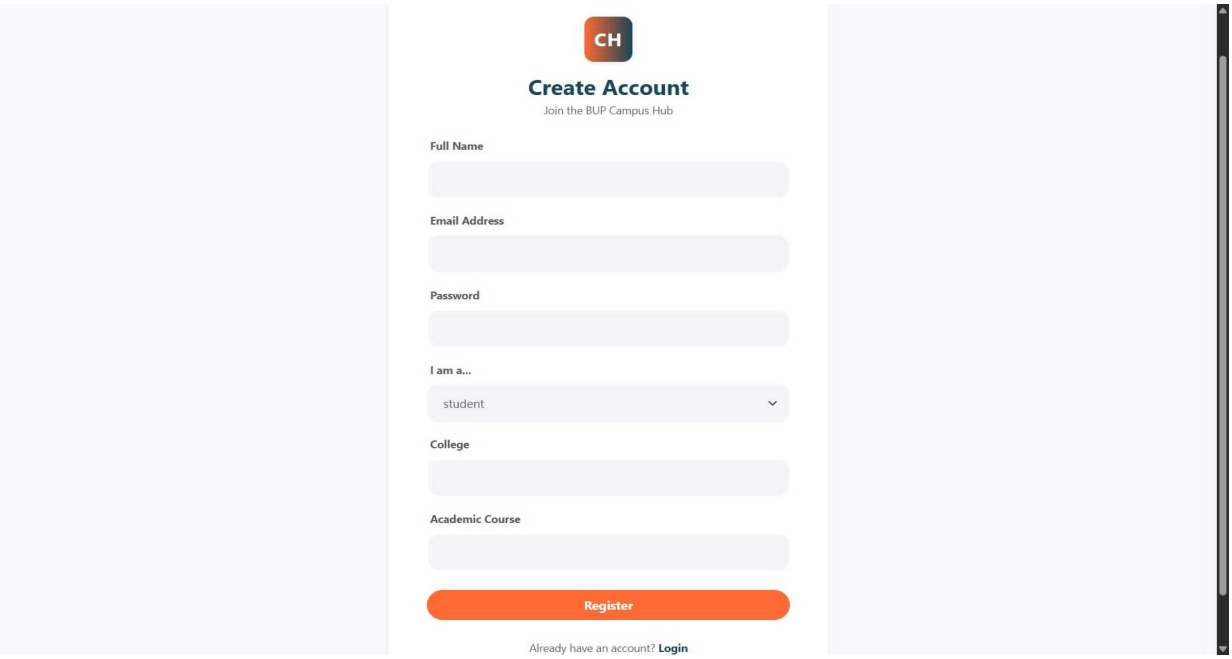
Task 5: GitHub Commits and File Structure

All updated and newly created files from this phase were committed to the GitHub repository under their respective directories. Frontend changes, including the updated HTML forms and new JavaScript handlers, were committed under /frontend, while any backend route adjustments were pushed under /backend. The GitHub Manager ensured that each commit carried a descriptive message that reflected the specific change made, such as "feat: Connected register form to backend API" and "fix: Resolved CORS issue on user POST route."

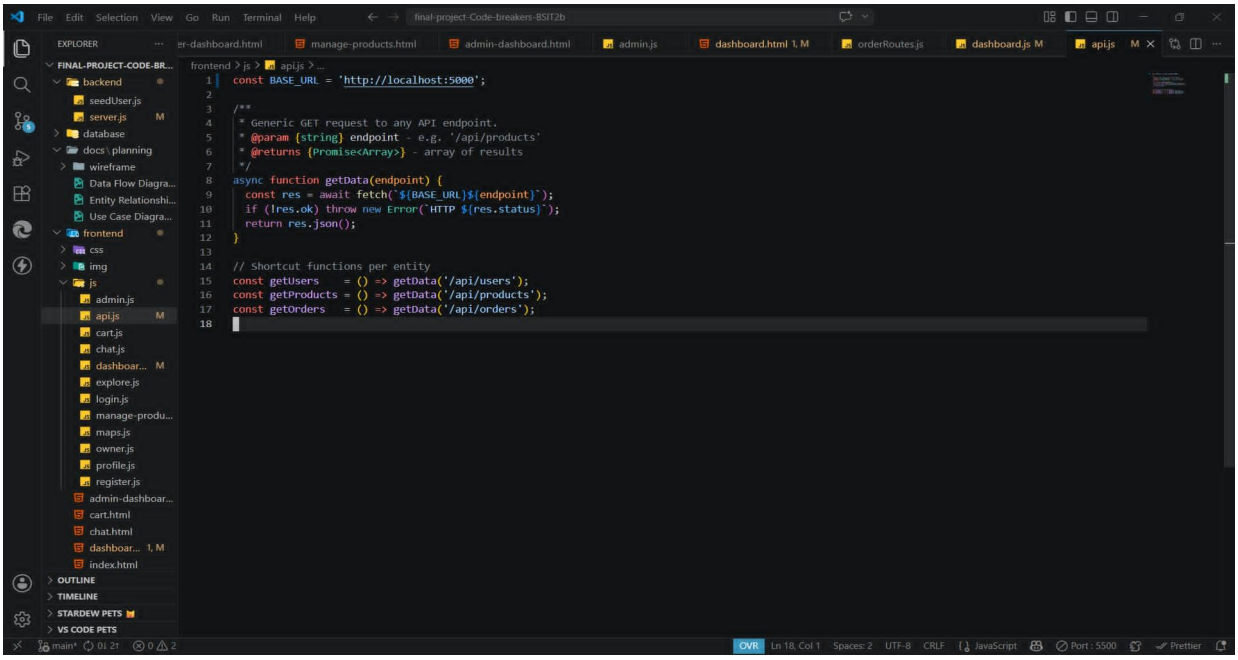
The repository structure was maintained consistently with the folder organization established in previous phases, preserving readability and traceability across the project's

development history. Screenshots of the commit log, folder structure, and MongoDB document insertions were documented and stored in the /docs/lab13/ directory for submission purposes.

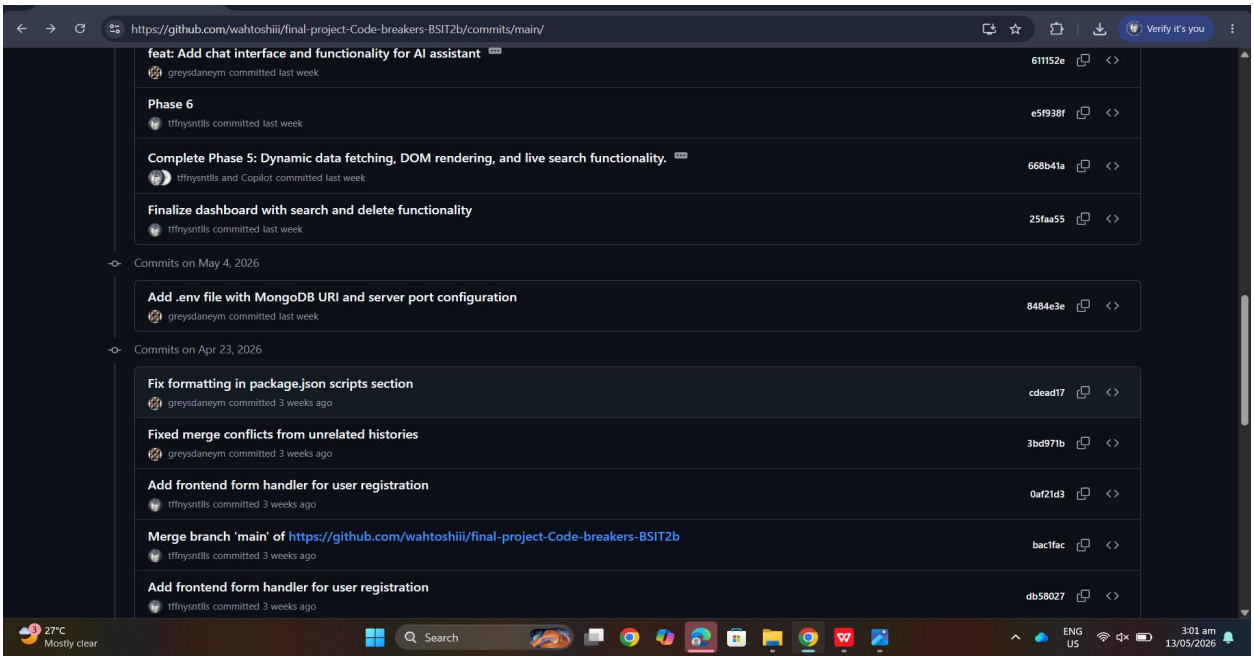
HTML Form Interface



JavaScript Code for API Connection



GitHub Commits



Contribution of Each Member

Carilla, Grace Ann S. – As the Backend Developer, I ensured that all POST endpoints were fully operational and correctly returning JSON responses before the frontend integration began. I also assisted in resolving a CORS configuration issue that initially prevented the browser from communicating with the local server, which was critical in making the fetch() calls work as intended.

Lamud, Ella Mae C. – During this phase, I was again unable to fulfill my role as GitHub Manager due to the continued unavailability of a personal laptop. My groupmates once more took on the responsibility of merging the updated frontend and backend branches, maintaining the folder structure, and ensuring that all commits were properly documented. I take full accountability for my absence and remain grateful for my team's willingness to carry the workload in my place.

Olarcos, Joshua L. – As the Project Manager, I coordinated the team's workflow during the integration phase, making sure that the frontend and backend developers were aligned in their implementation. I also assisted in debugging fetch() request errors and validating that the data flow from the form to the database matched the structure defined in our ERD.

Requio, Rein Ashley S. – As the group's tester, I conducted live form submission tests using both valid and invalid inputs to evaluate the system's behavior under different conditions. I documented all test results with screenshots, including the MongoDB Atlas dashboard showing newly inserted records, and reported any issues encountered so they could be addressed before submission.

Rubio, Ma. Miscy M. – As the Database Manager, I monitored the MongoDB Atlas dashboard throughout the testing phase to confirm that submitted form data was being correctly inserted into the appropriate collections and in accordance with the defined schema. I also verified that sensitive credentials, including the database connection string, remained secured in the .env file and were not exposed in the codebase.

Sentillas, Tiffany C. – As the Frontend Developer and Documentation Officer, I finalized the HTML form structures across the relevant pages, ensuring that all input fields were properly attributed for JavaScript targeting. I also compiled this lab report, organized the required screenshots, and ensured that all documentation components were complete and accurately represented the work accomplished during this phase.